

J1

Fondamentaux du langage

Prendre en main l'environnement, manipuler les types de base, écrire ses premières conditions et boucles.

01-premiers-pas

02-controle-de-flux

Objectifs de la journée



Un environnement qui marche

VS Code + uv : le même Python pour tout le monde, en une commande.



Variables, types, f-strings

int, float, str, bool — et des affichages propres.



Conditions et boucles

if / elif / else, for, while : le barème d'imposition en code.



Les données de la formation

Engendrer les jeux fiscaux fictifs qui serviront toute la semaine.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Fil rouge : la fiche d'un contribuable.

Un dépôt git, un environnement, des données



Le dépôt

Modules numérotés, TP à compléter, projet final — tout est versionné.
Chaque matin : git pull.



L'environnement

uv sync lit pyproject.toml et installe Python 3.12 + les bibliothèques, aux versions exactes.



Les données

Engendrées localement, jamais versionnées : contribuables (NIF), déclarations TVA, paiements.



démarrage — 3 commandes

```
git clone <adresse-du-depot> formation-python && cd formation-python
uv sync
cd 00-data && python generate_data.py --auto
```

Variables et types de base



Pas de déclaration de type

nif = "100004512" suffit — le type vient de la valeur.



type(x) pour vérifier

str, int, float, bool : quatre types couvrent 90 % des cas.



Des nombres lisibles

45_000_000 — le tiret bas sépare les milliers.



"45" + 5 plante — tant mieux

On convertit explicitement : int("45") + 5.



tp_premiers_pas.py

```
# La fiche d'un contribuable
nif = "100004512"
raison = " ets diallo "
ca = 45_000_000      # GNF
taux_tva = 0.18
```

```
print(type(ca))
<class 'int'>
```

```
tva = ca * taux_tva
print(tva)
8100000.0    # float !
```

Chaînes de caractères et f-strings



Les saisies sont sales

`.strip()` retire les espaces, `.title()` remet les majuscules.



La f-string, réflexe n°1

`f"Contribuable {nif}"` — la variable directement dans le texte.



Formater les montants

`{ca:,}` insère des séparateurs de milliers.



Tout est méthode

`upper`, `lower`, `replace`, `startswith`... `dir(str)` pour explorer.



chaînes

```
raison = "  ets fatoumata diallo  "
propre = raison.strip().title()
print(propre)
Ets Fatoumata Diallo

ca, tva = 45_000_000, 8_100_000
print(f"CA : {ca:,} GNF")
CA : 45,000,000 GNF

# les virgules → espaces :
f"{ca:,}".replace(",", " ")
```

Conditions : le régime d'imposition

< 100 M

Forfait

100 – 500 M

Réel simplifié

≥ 500 M

Réel normal

CA annuel en GNF (règle simplifiée pour la formation)



L'indentation EST la syntaxe

4 espaces, toujours — le bloc, c'est le retrait.



elif enchaîne les cas

Le premier vrai gagne ; else ramasse le reste.



tp_bareme.py

```
def regime(ca: int) -> str:
    if ca < 100_000_000:
        return "Forfait"
    elif ca < 500_000_000:
        return "Réel simplifié"
    else:
        return "Réel normal"

regime(200_000_000)
'Reel simplifié'
```

Boucles — parcourir des déclarations



for parcourt une collection

Pas d'indice à gérer : `for ca in chiffres`.



while répète jusqu'à condition

Utile quand on ne connaît pas le nombre de tours.



La compréhension : filtrer + transformer

Une ligne remplace quatre — à consommer avec lisibilité.



boucles

```
cas = [45_000_000, 0, 620_000_000, -5_000]

invalides = 0
for ca in cas:
    if ca <= 0:
        invalides += 1

# compréhension : régimes des CA valides
regimes = [regime(ca) for ca in cas
            if ca > 0]
['Forfait', 'Réal normal']
```

Le fil rouge démarre

- 1 **Installer et vérifier** — `INSTALLATION.md`, puis `uv run python main.py`
- 2 **Engendrer les données** — `generate_data.py --auto` — regardez les CSV produits
- 3 **`tp_premiers_pas.py`** — la fiche contribuable, proprement formatée
- 4 **`tp_bareme.py`** — le régime d'imposition + comptage des CA invalides
- 5 **Pour les rapides** — la boucle `while` « en combien d'années au réel normal ? »



Fichiers du jour

```
01-premiers-pas/  
    tp_premiers_pas.py  
02-controle-de-flux/  
    tp_bareme.py  
00-data/sortie/*.csv
```



Bloqué 10 minutes ? Appelez. Les solutions arrivent ce soir dans [solutions/](#).

À retenir

- ✓ **L'environnement** — uv sync = le même Python pour tous ; c'est la base du travail en équipe.
- ✓ **Les types** — la valeur porte le type ; on convertit explicitement, jamais par magie.
- ✓ **f-strings** — le réflexe d'affichage — lisible, formatable, débogable avec `f"{x=}"`.
- ✓ **if / for** — le barème et le comptage : la règle métier devient du code qui se lit.
- ✓ **git pull** — demain matin, avant tout : les solutions du jour seront publiées.



Demain — J2 : fonctions et structures de données

Découper le calcul des pénalités en fonctions testables, et construire un mini-annuaire avec listes, dictionnaires et ensembles.